

Лабораторна робота №4: Автентифікація та підключення до бази даних

Мета: Навчитись реалізовувати автентифікацію користувачів. Навчитись реалізовувати підключення до бази даних для зберігання та управління даними.

Теоретичні відомості

Автентифікація гарантує, що лише авторизовані користувачі можуть отримати доступ до певних ресурсів або функцій програми. Автентифікація важлива для захисту даних користувача, забезпечення конфіденційності та запобігання несанкціонованому доступу до конфіденційного контенту. Методи автентифікації включають **JWT (JSON Web Tokens)** і **OAuth**, які забезпечують безпечні механізми входу для веб- і мобільних додатків.

Автентифікація у веб-додатках

Автентифікація - це процес перевірки особи користувача або системи. У веб-додатках це, як правило, включає в себе перевірку того, що користувач є тим, за кого себе видає, перш ніж надати йому доступ до певних ресурсів. Існує кілька методів обробки автентифікації, але **JWT** і **OAuth** є двома найбільш широко використовуваними технологіями.

1. JWT (JSON Web Tokens)

JWT - це відкритий стандарт, який використовується для безпечної передачі інформації між сторонами у вигляді JSON-об'єктів. У контексті веб-додатків JWT зазвичай використовується для **аутентифікації без статусу**, що означає, що серверу не потрібно зберігати жодних сесійних даних про користувача. Коли користувач входить в систему, сервер генерує JWT, який містить специфічну для користувача інформацію (наприклад, ідентифікатор і роль користувача). Цей маркер надсилається клієнту, який включає його в заголовок авторизації для кожного наступного запиту на аутентифікацію користувача.

Переваги JWT:

- **Без збереження стану на сервері:** Серверу не потрібно зберігати дані про сесії користувачів.
- **Масштабованість:** Підходить для додатків, які можуть мати кілька серверів або потребують підтримки мобільних пристроїв.
- **Безпека:** JWT токени підписуються секретним ключем, який гарантує, що дані не можуть бути підроблені.

2. OAuth (Open Authorization)

OAuth - це більш складний і гнучкий фреймворк, який дозволяє користувачам автентифікуватися, використовуючи свої облікові дані від сторонніх провайдерів, таких як Google, Facebook або GitHub. OAuth зазвичай використовується для **делегованого доступу**, коли користувачі можуть надавати іншим програмам дозвіл на доступ до своєї інформації, не повідомляючи свій пароль. Наприклад, коли ви входите в сервіс за допомогою свого облікового запису Google, OAuth дозволяє цьому сервісу аутентифікувати вас, не знаючи вашого пароля Google.

Переваги OAuth:

- **Акаунт стороннього сервісу:** Дозволяє користувачам входити, використовуючи свої існуючі акаунти з інших сервісів.
- **Рівні дозволів:** OAuth дозволяє надавати конкретні дозволи, наприклад, на читання даних або публікацію контенту, не розкриваючи конфіденційні облікові дані.

Автентифікація є ключовим елементом для будь-якого веб-додатку, особливо для тих, що пов'язані з даними користувача або персоналізованим досвідом. Інтеграція безпечних механізмів автентифікації, таких як JWT і OAuth, гарантує, що додаток залишається захищеним від несанкціонованого доступу.

Одним з популярних методів автентифікації користувача є використання сервісу **Firebase Authentication**.

Firebase Authentication — це хмарний сервіс, який спрощує додавання автентифікації в веб- та мобільні додатки, підтримуючи вхід через електронну пошту, пароль, Google, Facebook, Apple та інші провайдери. Завдяки простому SDK та API, розробники можуть інтегрувати автентифікацію без складної серверної логіки. Сервіс забезпечує високу безпеку, використовуючи хешування паролів та OAuth 2.0, а також підтримує вхід без паролів через SMS або email-лінк. Інтеграція з іншими сервісами Firebase дозволяє легко керувати доступом до даних, а багатфакторна автентифікація (MFA) підвищує захист облікових записів. Firebase автоматично оновлює токени доступу, забезпечуючи безперервний користувацький досвід, а його глобальна інфраструктура гарантує стабільну та швидку роботу навіть при великій кількості користувачів.

Використання баз даних у веб-додатках

Бази даних відіграють ключову роль у веб-додатках, дозволяючи зберігати, обробляти та управляти даними користувачів у реальному часі. Вони можуть бути реляційними (MySQL, PostgreSQL) або нереляційними (MongoDB, Redis), залежно від потреб проєкту. Реляційні бази даних використовують структуровані таблиці з чіткими зв'язками між даними, що добре підходить для складних систем. Нереляційні бази даних зберігають інформацію у вигляді документів, колекцій або ключ-значення, що забезпечує гнучкість та високу продуктивність. Веб-додатки часто потребують масштабування баз даних, що досягається через шардінг, реплікацію або використання хмарних рішень. Швидкість доступу до даних оптимізується за допомогою індексів, кешування та ефективного запитування. Використання баз даних з API або ORM (Object-Relational Mapping) значно спрощує взаємодію між серверною частиною та збереженими даними. Безпека даних забезпечується через автентифікацію, шифрування та контроль доступу на рівні користувачів. Багато веб-додатків використовують бази даних у поєднанні з serverless-технологіями або мікросервісною архітектурою для підвищення продуктивності. У сучасних веб-застосунках база даних є невід'ємною частиною, що впливає на стабільність, швидкодію та масштабованість системи.

Використання Firebase бази даних та її переваги

Firebase Database — це хмарне рішення від Google, яке дозволяє веб-додаткам зберігати та синхронізувати дані в реальному часі. Вона доступна у двох варіантах: Realtime Database, яка зберігає дані у форматі JSON, та Cloud Firestore, що підтримує більш гнучкі структури

колекцій і документів. Основною перевагою є автоматична синхронізація змін між клієнтами без потреби в ручному оновленні даних. Firebase Database масштабується автоматично, що дозволяє ефективно працювати навіть при великому навантаженні. Завдяки інтеграції з Firebase Authentication можна легко керувати доступом користувачів до даних. Використання Firestore забезпечує офлайн-доступ, що дозволяє додаткам продовжувати роботу навіть без підключення до інтернету. Вбудовані функції безпеки, такі як Firestore Security Rules, дозволяють гнучко налаштовувати права доступу до різних частин бази даних. Firebase Database усуває потребу в окремому сервері для управління даними, що зменшує витрати на інфраструктуру. Вона добре інтегрується з іншими сервісами Firebase, такими як хмарне зберігання, аналітика та хостинг. Завдяки простому API та SDK для різних платформ Firebase Database значно спрощує розробку сучасних веб-додатків.

Інтегрувати Firebase з інтерфейсними фреймворками, такими як React, Angular або Vue, просто. Firebase надає SDK (Software Development Kits), що можна використовувати у серверній частині додатку для взаємодії з сервісами Firebase. Клієнтська частина додатку взаємодіє з серверною частиною, яка у свою чергу інтегрована з базою даних Firebase. Клієнтська частина додатку отримує дані та динамічно оновлює інтерфейс користувача відповідно до отриманих даних з бази.

Порядок виконання роботи

Крок 1: Налаштування автентифікації користувача

Налаштування автентифікації в Firebase

Автентифікація має важливе значення для захисту додатків. На цьому кроці реалізуємо автентифікацію користувачів за допомогою **JSON Web Tokens (JWT)** за допомогою **серверної частини Node.js** та інтегруємо його з **React, Angular або Vue** на клієнтській частині.

1. Зареєструватись/Увійти на Firebase (<https://firebase.google.com/>)
2. Перейти до консолі Firebase Console.
3. Вибрати проект (або створити новий, якщо ще не створили).
4. В огляді проекту натиснути на іконку  Налаштування (верхній лівий кут) → Налаштування проекту.
5. Прокрутити вниз до розділу "Ваші програми".
6. Натиснути "Додати програму" → Виберіть Web (іконка `</>`).
7. Вибрати проект → **Створити** → **Автентифікація**.
8. Натиснути кнопку "Почати" та увімкніть **автентифікацію за допомогою електронної пошти/пароля**.

Крок 2: Підключення автентифікації до клієнтської частини

Додати наступний код реалізації в корінь раніше створеного компонента фреймворку, який дозволить користувачам реєструватися, входити в систему і отримувати доступ до захищених маршрутів API.



```
import { initializeApp } from "https://www.gstatic.com/firebasejs/10.8.0/firebase-app.js";
import {
  getAuth,
  createUserWithEmailAndPassword,
  signInWithEmailAndPassword,
  signOut,
  getIdToken
}
from "https://www.gstatic.com/firebasejs/10.8.0/firebase-auth.js";
// Firebase config (Replace with your credentials)
const firebaseConfig = {
  apiKey: "YOUR_API_KEY",
  authDomain: "YOUR_AUTH_DOMAIN",
  projectId: "YOUR_PROJECT_ID",
  storageBucket: "YOUR_STORAGE_BUCKET",
  messagingSenderId: "YOUR_MESSAGING_SENDER_ID",
  appId: "YOUR_APP_ID"
};
// Initialize Firebase const app = initializeApp(firebaseConfig);
const auth = getAuth(app);
```

Пояснення необхідних полів конфігурації:

- **apiKey:** Унікальний ідентифікатор, який використовується для автентифікації запитів до сервісів Firebase.
- **authDomain:** Домен автентифікації для Firebase Authentication (використовується для входу/реєстрації).
- **projectId:** Унікальний ідентифікатор вашого проєкту Firebase.
- **storageBucket:** URL-адреса сховища Firebase, що використовується для зберігання файлів.
- **messagingSenderId:** Ідентифікатор відправника для Firebase Cloud Messaging (push-сповіщення).
- **appId:** Унікальний ідентифікатор вашого додатка Firebase, який використовується для ініціалізації сервісів Firebase.
- **measurementId** (необов'язково): Використовується для Firebase Analytics для відстеження взаємодії користувачів, якщо Google Analytics увімкнено.

Конфігурацію Firebase (firebaseConfig) можна отримати з консолі Firebase Console:

1. Відкрити консоль управління базою даних.
2. Вибрати проєкт (або створити новий, якщо ще не створений).
3. В огляді проєкту натиснути на іконку  Налаштування (верхній лівий кут) → Налаштування проєкту.
4. Прокрутити вниз до розділу "Ваші програми".
5. Firebase покаже об'єкт конфігурації з необхідними властивостями.

Додавання функції реєстрації

Наступний код представляє приклад реалізації логіки реєстрації користувача на клієнтській частині:

```

// Sign Up
async function signUp() {
  const email = document.getElementById("email").value;
  const password = document.getElementById("password").value;
  try {
    const userCredential = await createUserWithEmailAndPassword(auth, email, password);
    alert("User created: " + userCredential.user.email);
  } catch (error) {
    alert("Error: " + error.message);
  }
}

```

Додавання функції входу в систему

Наступний код представляє приклад реалізації логіки логіну користувача на клієнтській частині:

```

// Login
async function login() {
  const email = document.getElementById("email").value;
  const password = document.getElementById("password").value;
  try {
    const userCredential = await signInWithEmailAndPassword(auth, email, password);
    alert("Logged in: " + userCredential.user.email);
  } catch (error) {
    alert("Error: " + error.message);
  }
}

```

Додавання функції перевірки поточного користувача

Наступний код представляє приклад того, як отримати поточного автентифікованого користувача на клієнтській частині:

```
import { getAuth, onAuthStateChanged } from "firebase/auth";

// Initialize Firebase Auth
const auth = getAuth();

// Listen for authentication state changes
onAuthStateChanged(auth, (user) => {
  if (user) {
    // User is signed in
    console.log("User ID:", user.uid);
    console.log("Email:", user.email);
    console.log("Display Name:", user.displayName);
    console.log("Profile Photo URL:", user.photoURL);
  } else {
    // No user is signed in
    console.log("No user is currently signed in.");
  }
});
```

Крок 2: Підключення до бази даних

Для налаштування Firebase Firestore та додавання даних через інтерфейс виконайте наступні кроки:

1. Перейдіть у [Firebase Console](#) та натисніть **"Додати проєкт"**.
2. Введіть назву проєкту, за потреби увімкніть Google Analytics та натисніть **"Створити проєкт"**.
3. У лівому меню перейдіть у розділ **"Firestore Database"** та натисніть **"Створити базу даних"**.
4. Виберіть **"Test mode"** (дозволяє всім користувачам читати та записувати дані) або **"Production mode"** (обмежує доступ).
5. Виберіть **region зберігання** бази даних та натисніть **"Увімкнути"**.
6. У вкладці **"Дані"** натисніть **"Start Collection"**, введіть назву (наприклад, "users") і натисніть **"Далі"**.
7. Введіть **ID документа** або залиште поле порожнім для автоматичного створення.

8. Додайте поля до документа:

- **Ім'я поля:** name, **Тип:** String, **Значення:** "John Doe"
- **Ім'я поля:** email, **Тип:** String, **Значення:** "johndoe@example.com"
- **Ім'я поля:** age, **Тип:** Number, **Значення:** 25

9. Натисніть **"Зберегти"**, щоб додати документ у базу даних.

Наведений нижче приклад коду представляє, як читати дані з бази даних Firebase у браузері за допомогою JavaScript:

```
import { getDatabase, ref, set, push } from "firebase/database";

// Initialize Firebase Realtime Database
const db = getDatabase();

// Reference to a specific path in the database
const userRef = ref(db, "users/" + "user123");

// Set data in the database
set(userRef, {
  username: "JohnDoe",
  email: "johndoe@example.com",
  age: 25
}).then(() => {
  console.log("Data written successfully!");
}).catch((error) => {
  console.error("Error writing data:", error);
});
```

Наведений нижче приклад коду представляє, як записати дані в базу даних Firebase у браузері за допомогою JavaScript:

```
import { getDatabase, ref, get, child } from "firebase/database";

// Initialize Firebase Realtime Database
const db = getDatabase();

// Reference to the data
const userRef = ref(db);

// Get data once
get(child(userRef, "users/user123")).then((snapshot) => {
  if (snapshot.exists()) {
    console.log("User Data:", snapshot.val());
  } else {
    console.log("No data found.");
  }
}).catch((error) => {
  console.error("Error reading data:", error);
});
```

Лабораторне завдання

Завдання 1: Реалізація автентифікації користувачів за допомогою Firebase на клієнтській частині

Опис:

Додати автентифікацію користувачів до веб-додатку, використовуючи Firebase Authentication. Реалізувати можливість реєстрації, входу та виходу користувача через електронну пошту та пароль.

Кроки:

1. Створити обліковий запис у Firebase Console та додати проект.
2. Налаштувати Firebase Authentication:
 - Перейти в **Firebase Console > Authentication > Sign-in method** та увімкнути метод автентифікації через email та пароль.
3. Імпортувати Firebase SDK і налаштувати автентифікацію.
4. Створити форми для реєстрації та входу користувачів:
 - Форма для реєстрації повинна включати поля для email і пароля.
 - Форма для входу має включати поля для email і пароля.
5. Реалізувати функціональність реєстрації та входу користувача за допомогою Firebase.
6. Додати функціональність виходу користувача із системи.
7. Додати механізм відстеження стану автентифікації для показу статусу входу користувача в додаток.

Завдання 2: Підключення Firebase бази даних та читання/запис даних з клієнтської частини

Опис:

Підключити Firebase Firestore до веб-додатку та реалізувати можливість читання та запису даних у базу.

Кроки:

1. Увімкнути Firestore в Firebase Console:
 - Перейти в **Firebase Console > Firestore Database > Create Database** та вибрати режим доступу (тестування чи виробництво).
2. Імпортувати Firebase SDK і налаштувати Firestore для роботи з базою даних.
3. Створити функцію для запису даних у Firestore, наприклад, для збереження інформації про користувача.

4. Реалізувати функцію для читання даних з Firestore, щоб отримати збережену інформацію.
5. Додати виклики цих функцій для перевірки працездатності: додавання даних до бази та отримання їх з бази.

Контрольні запитання

1. Що таке JWT (JSON Web Token) і як він використовується для автентифікації користувачів у веб-додатках?
2. Які основні відмінності між JWT та OAuth у контексті автентифікації користувачів?
3. Які сервіси можна використовувати для розгортання клієнтської та серверної частини веб-додатка, і які їх основні особливості?
4. Як працює процес автентифікації користувачів за допомогою Firebase, і які кроки необхідні для його налаштування?
5. Які основні кроки необхідні для безпечного зберігання токенів автентифікації у веб-додатку?

Вимоги до оформлення звіту

1. Увійти в папку проекту



```
cd username.github.io
```

2. Створити нову папку з назвою **lab4** і помістити всі файли створені під час виконання лабораторної роботи в дану папку.
3. Завантажити зміни у репозиторій



```
git add --all  
git commit -m "Commit lab"  
git push -u origin main
```

Варіанти завдань

Варіант 1

Веб-сайт ресторану

1. Створити форми реєстрації та входу користувачів на сайт та впровадити функції автентифікації.
2. Реалізувати можливість залишати відгуки про ресторан тільки автентифікованим користувачам.
3. Створити хмарну базу даних з інформацією про пункти меню. Відображати інформацію у таблиці меню завантаженою зі створеної бази даних.

Варіант 2

Веб-сайт організації онлайн-ігор

1. Створити форми реєстрації та входу користувачів на сайт та впровадити функції автентифікації.
2. Для автентифікованих користувачів реалізувати можливість залишати оцінку ігор (1-5 зірок).
3. Створити хмарну базу даних для зберігання інформації про ігри та турніри. Використовувати інформацію з бази даних для відображення на клієнтській частині.

Варіант 3

Веб-сайт платформи для навчальних курсів

1. Створити форми реєстрації та входу користувачів на сайт та впровадити функції автентифікації.
2. Для автентифікованих користувачів відображати форму відгуку про курс. Неавтентифіковані користувачі можуть переглядати список курсів, але не мають доступу до перегляду деталей курсу та можливості залишати відгуки.
3. Створити хмарну базу даних для зберігання інформації про курси та розклад. Використовувати інформацію з бази даних для відображення на клієнтській частині.

Варіант 4

Веб-сайт платформи для бронювання турів

1. Створити форми реєстрації та входу користувачів на сайт та впровадити функції автентифікації.
2. Для автентифікованих користувачів реалізувати можливість залишати відгуки про тури, які відображаються в окремому розділі сторінки туру. Також додати можливість оцінювання туру за 5-зірковою шкалою. Неавтентифіковані користувачі можуть

переглядати список турів, але можливість бронювання туру та залишення відгуків для них заблокована.

3. Створити хмарну базу даних для зберігання інформації про тури. Використовувати інформацію з бази даних для відображення на клієнтській частині.

Варіант 5

Веб-сайт онлайн-магазину книг

1. Створити форми реєстрації та входу користувачів на сайт та впровадити функції автентифікації.
2. Для автентифікованих користувачів реалізувати можливість оформлення замовлення товарів доданих до кошика.
3. Створити хмарну базу даних для зберігання інформації про книги з каталогу. Використовувати інформацію з бази даних для відображення на клієнтській частині.

Варіант 6

Веб-сайт платформи для оренди автомобілів

1. Створити форми реєстрації та входу користувачів на сайт та впровадити функції автентифікації.
2. Реалізувати можливість робити бронювання тільки для автентифікованих користувачів.
3. Створити хмарну базу даних для зберігання інформації про доступні для бронювання автомобілі.

Варіант 7

Веб-сайт платформи для оренди житла

1. Створити форми реєстрації та входу користувачів на сайт та впровадити функції автентифікації.
2. Для автентифікованих користувачів реалізувати можливість залишати відгуки про квартири, які вони орендували, і відображати ці відгуки під описом квартири.
3. Створити хмарну базу даних для зберігання інформації про квартири. Використовувати інформацію з бази даних для відображення на клієнтській частині.

Варіант 8

Веб-сайт платформи для онлайн-бронювання квитків на заходи

1. Створити форми реєстрації та входу користувачів на сайт та впровадити функції автентифікації.
2. Для автентифікованих користувачів реалізувати можливість оцінки події (1-5 зірок).
3. Створити хмарну базу даних для зберігання інформації про події. Використовувати інформацію з бази даних для відображення на клієнтській частині.

Варіант 9

Веб-сайт платформи для організації волонтерських ініціатив

1. Створити форми реєстрації та входу користувачів на сайт та впровадити функції автентифікації.
2. Для автентифікованих користувачів реалізувати можливість залишати оцінки про ініціативи (1-5 зірок).
3. Створити хмарну базу даних для зберігання інформації про ініціативи. Використовувати інформацію з бази даних для відображення на клієнтській частині.

Варіант 10

Веб-сайт онлайн-магазину спортивних товарів

1. Створити форми реєстрації та входу користувачів на сайт та впровадити функції автентифікації.
2. Для автентифікованих користувачів реалізувати можливість створення персональних списків бажаних товарів, які можна додавати до кошика при необхідності.
3. Створити хмарну базу даних для зберігання інформації про продукти та акції. Використати інформацію з бази даних для відображення на клієнтській частині.

Варіант 11

Веб-сайт платформи для пошуку робочих вакансій

1. Створити форми реєстрації та входу користувачів на сайт та впровадити функції автентифікації.
2. Для автентифікованих користувачів реалізувати можливість редагування профілю, зокрема додавання досвіду роботи та навичок.
3. Створити хмарну базу даних для зберігання вакансій. Використовувати інформацію з бази даних для відображення на клієнтській частині.

Варіант 12

Веб-сайт платформи для замовлення їжі

1. Створити форми реєстрації та входу користувачів на сайт та впровадити функції автентифікації.
2. Реалізувати можливість оформити замовлення тільки для автентифікованих користувачів, при цьому неавтентифіковані користувачі побачать повідомлення про необхідність входу для завершення замовлення.
3. Створити хмарну базу даних для зберігання пунктів меню. Використовувати інформацію з бази даних для відображення на клієнтській частині.

Варіант 13

Веб-сайт блогу для подорожей

1. Створити форми реєстрації та входу користувачів на сайт та впровадити функції автентифікації.
2. Реалізувати можливість коментування та вподобання статей тільки для автентифікованих користувачів.
3. Створити хмарну базу даних для зберігання статей. Використовувати інформацію з бази даних для відображення на клієнтській частині.

Варіант 14

Веб-сайт інтерактивної платформи для вивчення мов

1. Створити форми реєстрації та входу користувачів на сайт та впровадити функції автентифікації.
2. Реалізувати можливість відображення сторінки Мій прогрес тільки для автентифікованих користувачів.
3. Створити хмарну базу даних для зберігання інформації про уроки. Використовувати інформацію з бази даних для відображення на клієнтській частині.

Варіант 15

Веб-сайт платформи для вивчення історії через інтерактивні події

1. Створити форми реєстрації та входу користувачів на сайт та впровадити функції автентифікації.

2. Реалізувати можливість відображення сторінки Тестування тільки для автентифікованих користувачів, при цьому неавтентифіковані користувачі будуть перенаправлені на сторінку входу.
3. Створити хмарну базу даних для зберігання інформації про історичні події, питання і відповіді тестів. Використовувати інформацію з бази даних для відображення на клієнтській частині.

Варіант 16

Веб-сайт симулятора управління власним стартапом

1. Створити форми реєстрації та входу користувачів на сайт та впровадити функції автентифікації.
2. Реалізувати можливість відображення сторінки Мій стартап тільки для автентифікованих користувачів.
3. Створити хмарну базу даних для зберігання інформації про ринки та інвесторів. Використовувати інформацію з бази даних для відображення на клієнтській частині.

Варіант 17

Веб-сайт платформи для створення та управління персональними цілями

1. Створити форми реєстрації та входу користувачів на сайт та впровадити функції автентифікації.
2. Реалізувати можливість створення, відображення цілей та відображення прогресу тільки для автентифікованих користувачів.
3. Створити хмарну базу даних для зберігання інформації про створені цілі. Використовувати інформацію з бази даних для відображення на клієнтській частині.

Варіант 18

Веб-сайт платформи для віртуальних хакатонів та змагань із програмування

1. Створити форми реєстрації та входу користувачів на сайт та впровадити функції автентифікації.
2. Реалізувати можливість доступу до сторінки Мої проекти тільки для автентифікованих користувачів.
3. Створити хмарну базу даних для зберігання інформації про змагання та рейтинги. Використовувати інформацію з бази даних для відображення на клієнтській частині.

Варіант 19

Веб-сайт симулятора управління будівництвом міста

1. Створити форми реєстрації та входу користувачів на сайт та впровадити функції автентифікації.
2. Реалізувати можливість доступу додавати нові об'єкти та керувати ресурсами тільки для автентифікованих користувачів.
3. Створити хмарну базу даних для зберігання інформації про об'єкти інфраструктури та ресурси міста. Використовувати інформацію з бази даних для відображення на клієнтській частині.

Варіант 20

Веб-сайт симулятора космічної експедиції

1. Створити форми реєстрації та входу користувачів на сайт та впровадити функції автентифікації.
2. Реалізувати можливість доступу до перегляду власних подорожей тільки для автентифікованих користувачів.
3. Створити хмарну базу даних для зберігання інформації про корабель та експедиції. Використовувати інформацію з бази даних для відображення на клієнтській частині.

Варіант 21

Веб-сайт платформи для моніторингу здоров'я та фізичної активності

1. Створити форми реєстрації та входу користувачів на сайт та впровадити функції автентифікації.
2. Реалізувати можливість доступу до перегляду власних тренувань тільки для автентифікованих користувачів.
3. Створити хмарну базу даних для зберігання інформації про доступні тренувальні програми. Використовувати інформацію з бази даних для відображення на клієнтській частині.

Варіант 22

Веб-сайт платформи для онлайн-курсу з фотографії

1. Створити форми реєстрації та входу користувачів на сайт та впровадити функції автентифікації.

2. Реалізувати можливість доступу до розділів "Галерея" та "Мій прогрес" тільки для автентифікованих користувачів.
3. Створити хмарну базу даних для зберігання інформації про уроки. Використовувати інформацію з бази даних для відображення на клієнтській частині.

Варіант 23

Веб-сайт платформи для оренди спортивного обладнання

1. Створити форми реєстрації та входу користувачів на сайт та впровадити функції автентифікації.
2. Реалізувати можливість оформлювати оренду обладнання доданого у кошик і переглядати активні та минулі оренди тільки для автентифікованих користувачів.
3. Створити хмарну базу даних для зберігання інформації про обладнання. Використовувати інформацію з бази даних для відображення на клієнтській частині.

Варіант 24

Веб-сайт платформи для планування подорожей

1. Створити форми реєстрації та входу користувачів на сайт та впровадити функції автентифікації.
2. Реалізувати можливість бачити список запланованих та завершених подорожей і план витрат на подорож тільки для автентифікованих користувачів.
3. Створити хмарну базу даних для зберігання інформації про список популярних туристичних напрямків. Використовувати інформацію з бази даних для відображення на клієнтській частині.

Варіант 25

Веб-сайт платформи для створення та обміну рецептами

1. Створити форми реєстрації та входу користувачів на сайт та впровадити функції автентифікації.
2. Реалізувати можливість бачити список власних рецептів тільки для автентифікованих користувачів.
3. Створити хмарну базу даних для зберігання інформації про категорії страв. Використовувати інформацію з бази даних для відображення на клієнтській частині.